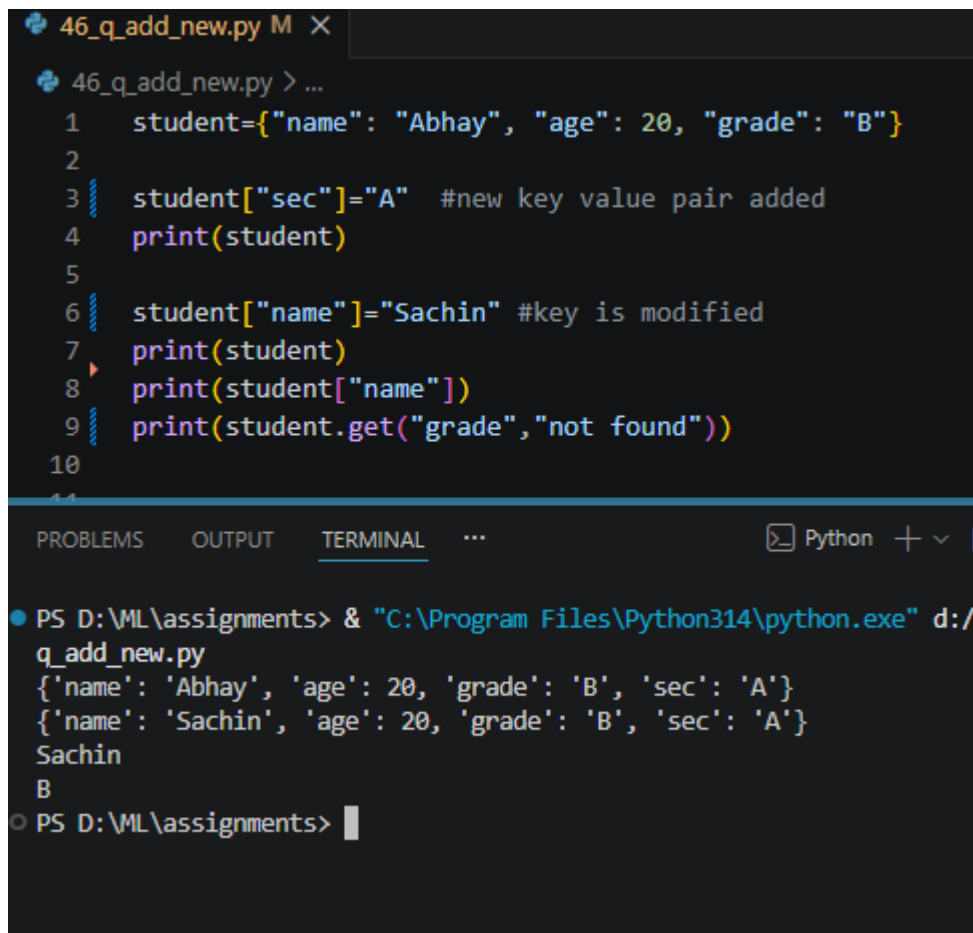


**Q1.** Write a python program to add a new key-value pair to a dictionary, modify an existing value and access a specific key.

- **Input:** student = {"name": "Abhay", "age": 20, "grade": "B"}



```
46_q_add_new.py M X
46_q_add_new.py > ...
1  student={"name": "Abhay", "age": 20, "grade": "B"}
2
3  student["sec"]="A" #new key value pair added
4  print(student)
5
6  student["name"]="Sachin" #key is modified
7  print(student)
8  print(student["name"])
9  print(student.get("grade","not found"))
10
11

PROBLEMS OUTPUT TERMINAL ... Python + v [
PS D:\ML\assignments> & "C:\Program Files\Python314\python.exe" d:/
q_add_new.py
{'name': 'Abhay', 'age': 20, 'grade': 'B', 'sec': 'A'}
{'name': 'Sachin', 'age': 20, 'grade': 'B', 'sec': 'A'}
Sachin
B
PS D:\ML\assignments> |
```

**Q2.** Write a python program to remove a specific key from a dictionary, retrieve all key-value pairs and check whether a given key exists.

- **Input:** car = {"brand": "Toyota", "model": "Camry", "year": 2022, "color": "blue"}

```
47_remove_retrieve_check_dict.py > ...
1  car={
2      "brand": "Toyota",
3      "model": "Camery",
4      "year": "2022",
5      "color": "blue"
6  }
7
8  print(car)
9  del_item=car.pop("brand") #delete specific key from dict
10 print(car)
11
12
13 for i in car.items(): #retrive all key value pairs
14     print(i)
15
16 #check whether a given key exists
17 exists_key="model"
18 exists= exists_key in car
19
20 print(f"{exists_key} exists in car?: {exists}")
21
```

PROBLEMS OUTPUT TERMINAL ... Python + - [] [] ...

```
● PS D:\ML\assignments> & "C:\Program Files\Python314\python.exe" d:/ML/assignme
remove_retrieve_check_dict.py
{'brand': 'Toyota', 'model': 'Camery', 'year': '2022', 'color': 'blue'}
{'model': 'Camery', 'year': '2022', 'color': 'blue'}
('model', 'Camery')
('year', '2022')
('color', 'blue')
model exists in car?: True
○ PS D:\ML\assignments> []
```

**Q3.** Write a python program to create a dictionary by mapping two equal-length lists, one containing keys and the other containing values.

- **Input:** keys = ["name", "age", "city"] and values = ["Abhay", 20, "Jharkhand"]

```

1 keyss=["name", "age", "country"]
2 values=["Abhay", 20, "india"]
3
4 emp_dict={}
5 for i in range(0, len(keyss)):
6     print(keyss[i])
7     emp_dict[keyss[i]]=values[i]
8 print(emp_dict)
9

```

PROBLEMS OUTPUT TERMINAL ...

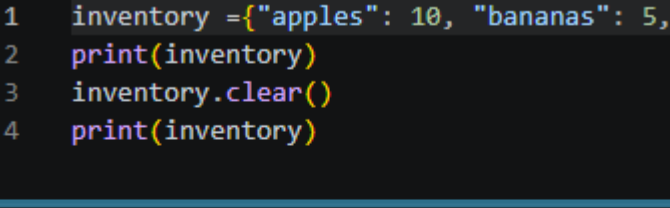
```

PS D:\VML\assignments> & "C:\Program Files\Python314\python.exe" d:\
create_dict_using_two_eq_len_list.py
name
age
country
{'name': 'Abhay', 'age': 20, 'country': 'india'}
PS D:\VML\assignments>

```

**Q4.** Write a python program to remove all items from a dictionary while keeping the dictionary object itself intact.

- **Input:** inventory = {"apples": 10, "bananas": 5, "oranges": 8}



The screenshot shows a code editor with a Python script and its terminal output. The script, named `49_clear_dict_keeping_object.py`, defines a dictionary `inventory` with keys `"apples"`, `"bananas"`, and `"oranges"` and values `10`, `5`, and `8` respectively. It then prints the dictionary, clears it, and prints it again. The terminal output shows the initial dictionary, followed by an empty dictionary after the `clear()` method is called.

```
49_clear_dict_keeping_object.py > inventory
1  inventory = {"apples": 10, "bananas": 5, "oranges": 8}
2  print(inventory)
3  inventory.clear()
4  print(inventory)
```

TERMINAL

```
PS D:\ML\assignments> & "C:\Program Files\Python314\python.exe" d:/ML/assignments/49_clear_dict_keeping_object.py
{'apples': 10, 'bananas': 5, 'oranges': 8}
{}
PS D:\ML\assignments>
```

**Q5.** Write a python program to combine two dictionaries into a single dictionary. If both dictionaries share a key, the value from the second dictionary should take precedence.

- **Input:** dict1 = {"a": 1, "b": 2} and dict2 = {"b": 3, "c": 4}

```

50_adding_two_dict_with_precedence.py > ...
1 dict1={"a":1, "b":2}
2 dict2={"b":3, "c":4}
3
4 dict1.update(dict2)
5
6 print(dict1)
7

TERMINAL ... Python + v []
PS D:\ML\assignments> & "C:\Program Files\Python314\python.exe" d:/ML/assignments/50_adding_two_dicts.py
{'a': 1, 'b': 3, 'c': 4}
PS D:\ML\assignments>

```

**Q6.** Write a python program to retrieve a specific value from a dictionary that is nested inside another dictionary.

- **Input:** person = {"name": "Abhay", "address": {"city": "Jharkhand", "zip": "75701"}}

```

51_retrieve_nested_dict.py > [?] person
1 person={"name": "Abhay", "address": {"city": "jarkhand", "zip": "75701"}}
2 print(person["address"]["city"])
3 print(person["address"]["zip"])

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + v []
● PS D:\ML\assignments> & "C:\Program Files\Python314\python.exe" d:/ML/assignments/51_retrieve_nested_dict.py
jarkhand
75701
PS D:\ML\assignments>

```

**Q7.** Write a python program to access the value associated with the key 'History' from a dictionary nested within the larger data structure.

- **Input:** student = {"name": "Abhay", "grades": {"math": 88, "Science": 92, "History": 75}}

```
52_accessing_nested_dict.py > ...
1  student={
2      "name": "Abhay",
3      "grade": {
4          "math": 88,
5          "Science": 12,
6          "history": 75
7      }
8  }
9
10 print(student["grade"]["history"])
11
```

PROBLEMS TERMINAL ... Python + - []

```
PS D:\ML\assignments> & "C:\Program Files\Python314\python.exe" /ML/assignments/52_accessing_nested_dict.py
75
PS D:\ML\assignments>
```

**Q8.** Write a python program to create a dictionary from a list of keys, assigning the same default value 0 to every key.

- **Input:** keys = ["math", "Science", "English", "History"] and default = 0

```
53_create_dict_from_list_of_keys_and_val_0.py > keys
1  keys=["Math","Science","English","history"]
2  empty_dict={}
3  for i in keys:
4      empty_dict[i]=0
5  print(empty_dict)
6
```

PROBLEMS TERMINAL ... Python + - [] ...

```
PS D:\ML\assignments> & "C:\Program Files\Python314\python.exe" /ML/assignments/53_create_dict_from_list_of_keys_and_val_0.py
{'Math': 0, 'Science': 0, 'English': 0, 'history': 0}
PS D:\ML\assignments>
```